

SUPERSYSTEMS PostgreSQL Schema Guide

A Study Guide & 100 SQL Interview Questions for Senior Data Engineers (5+ Years Exp)

Part 1: Relational Schemas & Logic Explanations

- **Part Management (`part`)**: Registry for universal item codes (`part.parts\_master` containing `part\_number`), separator rules, category serial counters, and cross-reference catalog tables (`part.customer\_mappings`).
- **Purchase & Procurement**: Requisition workflows (`purchase.purchase\_requisitions`) and PO lead time tracker rules (`procurement.purchase\_orders`) storing pricing, MOQ constraints, and promised dates.
- **Inventory (`inventory`)**: Tracks stock levels (`qty\_on\_hand`, `qty\_reserved`, `qty\_available`) and keeps an immutable transaction history ledger of all ISSUE, RECEIPT, or TRANSFER operations.
- **Warehouse (`warehouse`)**: Physical layouts including subdivisions like zones and bins (`bin\_code`, `zone\_code`, `capacity\_units`) and receiving logs.
- **Quality Management (`quality`)**: QC inspection parameters (`quality.iqc\_criteria`) and non-conformance records (`quality.ncrs`) generated automatically on rejections.

Example A: Raw Material Shortage & MOQ Calculations Query

```
SELECT
  b.item_code,
  b.qty_required - COALESCE(s.qty_available, 0) AS raw_shortage,
  GREATEST(
    b.qty_required - COALESCE(s.qty_available, 0),
    q.min_order_qty
  ) AS final_qty_to_order
FROM manufacturing.bom_lines b
LEFT JOIN inventory.stock_levels s ON b.item_code = s.part_number
LEFT JOIN purchase.supplier_quotations q ON b.item_code = q.part_or_rm_code
WHERE b.qty_required > COALESCE(s.qty_available, 0);
```

Example B: Quality IQC Approval updates available stock levels

```
-- Step 1: Update Live Stock Levels
UPDATE inventory.stock_levels
SET qty_on_hand = qty_on_hand + :passed_qty,
    qty_available = qty_available + :passed_qty,
    updated_at = NOW()
WHERE part_number = :part_code AND bin_code = :bin_code;

-- Step 2: Auto-create NCR if rejected_qty > 0
INSERT INTO quality.ncrs (id, ncr_no, defect_type, severity, corrective_action, tenant_id)
VALUES (:ncr_id, :ncr_no, 'IQC Rejection', 'Major', 'Return to Vendor', :tenant_id);
```

Part 2: 100 Data Engineer SQL Interview Questions

1. Schema Design & Data Modeling (Questions 1 - 20)

- Q1: What is the difference between Star Schema and Snowflake Schema? How would you design a Snowflake representation for the `warehouse` bin hierarchy?
- Q2: Explain the concept of Slowing Changing Dimensions (SCD) Types 1, 2, and 3. How would you model historical price change revisions in `purchase.supplier\_quotations` using SCD Type 2?
- Q3: What is a surrogate key, and when is it preferred over a natural composite key? Consider `parts\_master.part\_number` vs a UUID.
- Q4: How do you handle many-to-many relationships in relational databases? Give an example using projects and purchase orders.
- Q5: What are normal forms (1NF, 2NF, 3NF, BCNF)? Explain why the jsonb `lines` column in `procurement.purchase\_orders` violates 1NF and the trade-offs of storing it this way.
- Q6: What is denormalization, and in what scenarios does it improve read performance?
- Q7: How do you design an audit logging schema like `audit.logs` to capture changes (insert/update/delete) for sensitive tables without using triggers?
- Q8: Explain the trade-offs of using UUIDs vs Auto-incrementing BigInts as primary keys in PostgreSQL.
- Q9: How would you model hierarchical parent-child relationships, such as parent accounts in `finance.accounts`?
- Q10: Describe the concept of Partitioning in PostgreSQL. How would you partition the `inventory.stock\_movements` table by range (e.g., monthly)?
- Q11: What is a write-ahead log (WAL) and how does it guarantee transaction durability?
- Q12: What is Referential Integrity? Discuss the implications of `ON DELETE CASCADE` vs `ON DELETE SET NULL` on the `project.projects` and `project.tasks` relationship.
- Q13: How does database sharding differ from partition partitioning?
- Q14: How would you store unstructured or semi-structured data in PostgreSQL (JSON vs JSONB)?
- Q15: What are indexing strategies for JSONB columns in PostgreSQL? Explain GIN index vs BTREE index.
- Q16: Describe the difference between OLTP and OLAP database engines. Which category does this SUPERSYSTEMS PostgreSQL instance fit into?
- Q17: Explain the Entity-Attribute-Value (EAV) design pattern. What are its pros and cons?
- Q18: How would you model user roles and permissions (RBAC) like the `iam.module\_access` table to allow granular column-level control?
- Q19: Discuss the design of a database schema that supports multi-tenancy. What are the pros and cons of shared database (logical separation via `tenant\_id`) vs schema-per-tenant?
- Q20: What is a metadata schema? How would you design a table to dynamically define UOM conversions across different modules?

2. Performance Tuning & Indexing (Questions 21 - 40)

- Q21: What is an Index? How does a B-Tree index work under the hood?
- Q22: Explain the difference between Clustered and Non-Clustered indexes.
- Q23: What is a Composite Index? Why does the order of columns in a composite index matter?
- Q24: How do you identify slow-running queries in PostgreSQL? Explain the configuration parameters like `pg_stat_statements`.
- Q25: Explain the output of `EXPLAIN ANALYZE`. What is the difference between a Sequential Scan, Index Scan, and Index Only Scan?
- Q26: What is a Index Only Scan, and what conditions are required to trigger it?
- Q27: What is index selectivity, and how does it impact the query planner's decision to use an index?
- Q28: Explain what database statistics are and why running `ANALYZE` regularly is critical for query optimizer performance.
- Q29: What is Index Bloat? How do you detect and resolve index bloat in PostgreSQL?
- Q30: Explain the difference between `VACUUM`, `VACUUM FULL`, and `autovacuum`.
- Q31: What is a Partial Index? Write a query to create a partial index on `inventory.stock_levels` for rows where `qty_available <= reorder_point`.
- Q32: What is an Expression Index? How would you index case-insensitive email lookups in `iam.users`?
- Q33: Explain the term "Parameter Sniffing" and how it affects query plan caching.
- Q34: What is a Hash Join, Merge Join, and Nested Loop? In what conditions does the query planner choose one over another?
- Q35: How does the size of the database buffer cache (`shared_buffers` in PG) affect index scan performance?
- Q36: How do you optimize queries that perform large `LIMIT` and `OFFSET` pagination? Suggest alternative cursor-based pagination strategies.
- Q37: What is a covering index, and how does the `INCLUDE` clause help in index-only scans?
- Q38: Explain what happens during a table scan when a table is heavily fragmented.
- Q39: How do you tune queries that contain complex OR conditions in their `WHERE` clause to ensure they utilize indexes?
- Q40: Explain the difference between active indexing and writing indexes on write-heavy tables.

3. Advanced Analytical SQL (Questions 41 - 70)

- Q41: What are Window Functions? Write a query using `ROW_NUMBER()`, `RANK()`, and `DENSE_RANK()` to rank products by inventory stock value.
- Q42: Write a query to calculate a 3-month moving average of total purchase order amounts from `procurement.purchase_orders`.
- Q43: How do you find the running total of stock movements for a specific item over time using window functions?

- Q44: Explain the difference between `ROWS BETWEEN` and `RANGE BETWEEN` in window function frame specifications.
- Q45: Write a recursive CTE query to display the entire hierarchy of accounts starting from the root parent in `finance.accounts`.
- Q46: Write a recursive CTE query to resolve a multi-level Bill of Materials (BOM) explosion, aggregating components needed for a parent finished good.
- Q47: What is a Common Table Expression (CTE)? When is a CTE preferred over a subquery, and what is its performance impact in PostgreSQL (materialized vs non-materialized)?
- Q48: What are `LEAD()` and `LAG()` window functions? Write a query to find the lead time difference in days between consecutive purchase orders for each supplier.
- Q49: Write a query using `COALESCE` and `NULLIF` to prevent division-by-zero errors when calculating the order fill rate (received qty / requested qty).
- Q50: Explain `PIVOT` and `UNPIVOT` concepts. How would you pivot the weekly total purchase order counts by status in PostgreSQL using `FILTER` clauses?
- Q51: What is the difference between `UNION` and `UNION ALL`? Explain their memory and performance characteristics.
- Q52: Write a query to identify top 5% of customers by total invoice amount using `NTILE()`.
- Q53: How do you perform fuzzy string matching in SQL? Explain `LIKE`, `ILIKE`, Levinshtein distance, and Trigrams.
- Q54: Write a query to group and aggregate sales data using `GROUP BY CUBE` and explain its difference from `GROUP BY ROLLUP` and `GROUP BY GROUPING SETS`.
- Q55: Explain the execution order of clauses in a SQL statement (`FROM`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY`, `LIMIT`).
- Q56: What is a correlated subquery? Write a query to find all employees whose salary is above the average salary of their department.
- Q57: Write a query to extract, parse, and aggregate elements from the JSONB column `lines` in the `procurement.purchase_orders` table.
- Q58: Explain the difference between `CROSS JOIN` and `INNER JOIN`. In what real-world analytics cases is a cross join useful?
- Q59: How would you write a query to find the median value of purchase order totals without using built-in median functions in databases that do not support it?
- Q60: Write a query to identify overlapping project task schedules in `project.tasks` using interval logic.
- Q61: What is the difference between `ANY`, `ALL`, and `SOME` operators in subquery evaluations?
- Q62: Write a query using `EXISTS` vs `IN` to check if supplier parts have active contracts. Explain the performance differences.
- Q63: Explain the difference between `NULL` and empty string `''`. How does SQL evaluate expressions like `NULL = NULL` and `NULL IS NULL`?
- Q64: Write a query to calculate the Year-Over-Year (YoY) growth of invoice amounts.

Q65: What is the `LATERAL` join operator in PostgreSQL? Write a query using `LEFT JOIN LATERAL` to fetch the 3 most recent stock movements for each warehouse bin.

Q66: How do you implement date-binning or grouping by arbitrary time intervals (e.g., every 15 minutes) for timeseries data?

Q67: Explain the function of the `CASE` statement. Write a query categorizing stock levels into 'Out of Stock', 'Reorder Alert', and 'Healthy' buckets.

Q68: Write a query to find the first non-null contact method (mobile, landline, email) for employees.

Q69: What is a self-join? Write a query to list all employees and the names of their direct managers from `hr.employees`.

Q70: Write a query to identify gaps in consecutive document sequences (e.g., missing numbers in invoice document sequence `INV-001`, `INV-003`).

4. Transactions, Concurrency & Security (Questions 71 - 85)

Q71: What does the ACID acronym stand for? Explain each concept in detail.

Q72: What are the four ANSI SQL transaction isolation levels? What concurrency anomalies (Dirty Read, Non-repeatable Read, Phantom Read) does each prevent?

Q73: What is Read Committed isolation level? Explain how dirty reads are avoided.

Q74: Explain MVCC (Multi-Version Concurrency Control) in PostgreSQL. How does it manage concurrent reads and writes without locking?

Q75: What is a Deadlock? How does a database engine detect and resolve deadlocks, and how can you write queries to prevent them?

Q76: What is the difference between Shared Locks (S) and Exclusive Locks (X)?

Q77: Explain the use of the `FOR UPDATE` clause in a `SELECT` query. When should you use `FOR UPDATE SKIP LOCKED`?

Q78: What is Write-Ahead Logging (WAL) and how does it guarantee transaction durability?

Q79: Explain the difference between a database transaction `COMMIT` and `ROLLBACK` at the disk/page level.

Q80: Describe SQL Injection and write a code example showing how to prevent it using parameterized queries.

Q81: What is Role-Based Access Control (RBAC) vs Row-Level Security (RLS) in PostgreSQL? How would you write an RLS policy for the `inventory.stock_levels` table to restrict records by `tenant_id`?

Q82: What are Database Triggers? Explain the difference between `BEFORE` and `AFTER` triggers, and row-level vs statement-level triggers.

Q83: What is a savepoint inside a transaction? Write a transaction script using savepoints to partially roll back changes on validation failure.

Q84: Explain the difference between `TRUNCATE` and `DELETE`.

Q85: How do you handle schema upgrades in production databases safely without causing table locks and downtime?

5. ETL, ELT & Data Pipelines (Questions 86 - 100)

Q86: What is the difference between ETL (Extract, Transform, Load) and ELT (Extract, Load, Transform)? When is one preferred over the other?

Q87: What is Change Data Capture (CDC)? How does log-based CDC differ from query-based CDC?

Q88: Explain the concept of Idempotency in data pipelines. Why is it important, and how do you design an idempotent insert pipeline?

Q89: How do you handle backfills of historical data in a production data pipeline without impacting live transaction tables?

Q90: Explain the difference between Batch processing and Stream processing.

Q91: What is Data Lineage, and why is it crucial for data engineering platforms?

Q92: How do you handle late-arriving data in a time-series data warehouse?

Q93: What is the difference between a database view, a materialized view, and a table?

Q94: Explain the concept of a Staging Area in ETL architecture.

Q95: How do you handle schema evolution (e.g., columns being added or data types changing) in upstream sources without breaking downstream tables?

Q96: Explain what a surrogate key is in a Data Warehouse. Why is it preferred over natural keys for business reporting?

Q97: What is Upsert? Write a PostgreSQL SQL statement using `INSERT ... ON CONFLICT DO UPDATE` to upsert stock levels.

Q98: What is the role of metadata databases (e.g., Apache Airflow DB) in orchestrating data workflows?

Q99: Explain how you would monitor database query performance trends over time using ELK stack or Prometheus/Grafana.

Q100: How do you design a data validation pipeline to check for referential integrity, null values, and outliers before loading data into a BI data warehouse?